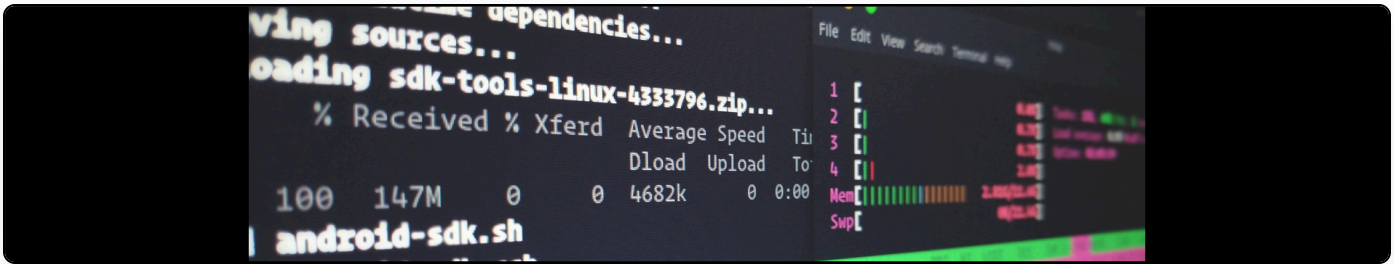
Overriding hypothesis's interface the right way

Cite as: Martin Paul Eve, "Overriding hypothesis's interface the right way", *eve.gd: Martin Paul Eve*, May 03, 2016, https://doi.org/10.59348/ya401-nc181.



Hypothes.is is an annotation framework that uses Pyramid to provide interface asset locations. This means that it is possible to override the interface and other components with one's own assets without simply forking the main hypothes.is repository.

However, it's not actually completely clear how one does this. There aren't any code samples yet that show it and the directions in forum posts are somewaht cryptic and require an intricate knowledge of the Pyramid framework's asset-handling and substitution facilities.

It turns out that the basic steps are as follows:

- First, clone the "h" repo from hypothes.is
- Second, create your own app structure, including your templates directory

Then, inside the module that runs when your codebase starts, we need to setup a series of methods that will instantiate the hypothes.is application. A rough template for this looks as follows:

```
from pyramid.config import Configurator
from h.assets import *
from h.config import settings_from_environment

def includeme(config):
    config.registry.settings.setdefault('webassets.bundles', 'annotran:assets.yaml')
    config.include('pyramid_webassets')
    config.override_asset(
        to_override='h:templates/old-home.html.jinja2',
        override_with='myproject:templates/home.html.jinja2')
    config.commit()

def get_settings(global_config, **settings):
    result = {}
    result.update(settings)
    result.update(settings_from_environment())
    return result


def main(global_config, **settings):
    settings = get_settings(global_config, **settings)
    config = Configurator(settings=settings)

    config.set_root_factory('h.resources.create_root')
    config.add_subscriber('h.subscribers.add_renderer_globals',
                      'pyramid.events.BeforeRender')
    config.add_subscriber('h.subscribers.publish_annotation_event',
                      'h.api.events.AnnotationEvent')
    config.add_tween('h.tweens.conditional_http_tween_factory',
under='pyramid.tweens.excview_tween_factory')
    config.add_tween('h.tweens.csrf_tween_factory')
    config.add_tween('h.tweens.auth_token')
    config.add_renderer('csv', 'h.renderers.CSV')
    config.include(__name__)
    config.include('h.app')
    return config.make_wsgi_app()
```

When this code is run the sequence is as follows:

- The main function fires. You should pass this a dictionary of setup parameters. You can see what this looks like by putting a breakpoint on the same method inside hypothes.is.

- The main function establishes a hypothes.is app. Because we've put the line "config.include(__name__)" though, this module will be included when the Pyramid framework loads.
- When the application is loading, the includeme function in this module will be called by Pyramid.
- The "includeme" function here says "instead of using h:templates/old-home.html.jinja2 from hypothes.is, use myproject:templates/home.html.jinja2 from myproject".

Tada. You now have overriding of hypothes.is templates in a way that doesn't break the external library dependency or create an unnecessary fork.

My thanks to Marija Katic for working this out with me!